Data Scraping & Trust Scoring System

Introduction

The goal of this system is to automatically collect content from multiple sources and evaluate how trustworthy that content is. It combines web scraping, natural language processing (NLP), and a rule-based scoring system to simulate how an intelligent system might judge the credibility of information.

Objective

The main objectives of this project are:

- To extract structured data from different types of sources (blogs, YouTube, PubMed)
- To process and analyze the content using NLP techniques
- To design a trust scoring system that evaluates the reliability of each source
- To store the processed data in a structured and reusable format

Data Sources

The system collects data from three different platforms:

1. Blogs

We scrape blog articles to extract:

- Main content
- Author name
- Publication date

2. YouTube Videos

For YouTube, the system extracts:

- Channel name
- Video description
- Publish date
- Transcript (if available)

3. PubMed Articles

From PubMed, we extract:

- Research paper title
- Authors
- Journal name
- Abstract
- Publication year

This combination ensures that the system works with both informal (blogs/videos) and formal (research papers) content.

4. System Architecture

The project follows a modular structure:

```
project/
│
├── scraper/
│   ├── blog_scraper.py
│   ├── youtube_scraper.py
│   ├── pubmed_scraper.py
│
├── utils/
│   ├── text_processing.py
│   ├── tagging.py
│   ├── chunking.py
```

```
│   ├── language.py
│
├── scoring/
│   └── trust_score.py
│
├── output/
│   ├── blogs.json
│   ├── youtube.json
│   ├── pubmed.json
│
├── main.py
└── app.py
```

Scraping Approach

Blog Scraping

I've use BeautifulSoup to parse HTML pages. The system removes unwanted elements like ads and navigation menus to extract only meaningful article content.

YouTube Scraping

I've use the YouTube Transcript API to extract subtitles. This allows us to analyze the actual spoken content of the video instead of relying only on titles or descriptions.

PubMed Scraping

I've use Biopython's Entrez API to fetch structured research data such as abstracts and author information.

Content Processing (NLP)

After scraping, the content goes through multiple processing steps:

1. Language Detection

The system identifies the language of the content automatically. This ensures the system can handle multilingual data.

2. Topic Tagging

Important keywords are extracted from the text to understand the main topic.

Example tags:

- "Machine Learning"
- "Healthcare"
- "Artificial Intelligence"

3. Content Chunking

Long content is divided into smaller chunks (paragraphs or transcript segments).

This makes it easier for downstream processing and analysis.

Data Structure

Each source is stored in a structured JSON format:

```
{
  "source_url": "",
  "source_type": "",
  "author": "",
  "published_date": "",
  "language": "",
  "region": "",
  "topic_tags": [],
  "trust_score": "",
```

```
  "content_chunks": []
}
```

The data is saved into:

- `blogs.json`
- `youtube.json`
- `pubmed.json`

Trust Score System

The most important part of this project is the trust scoring system.

Instead of using a complex machine learning model, we use a **rule-based approach**, which is easier to understand and explain.

◇ Factors Considered

The trust score is calculated based on:

- **Author Credibility**

If the author is present, the score increases

- **Domain Authority**

PubMed > Blogs > YouTube (based on reliability)

- **Recency**

Newer content is considered more reliable

- **Content Quality**

Based on length and structure

- **Citation Count (Approximated)**

Research papers are given higher weight

- **Medical Disclaimer (Heuristic)**

Health-related content without disclaimers is slightly penalized

◇ Final Score

The final trust score is normalized between:

👉 **0 (low trust) to 1 (high trust)**

Edge Case Handling

Real-world data is messy, so the system handles:

- Missing author → reduces score
- Missing publish date → reduces score
- No YouTube transcript → handled gracefully
- Non-English content → still processed
- Very long content → split using chunking

⍰ Abuse Prevention Logic

To prevent manipulation and fake content:

- Unknown or fake authors are penalized
- Low-quality domains reduce score
- Outdated content is penalized
- Medical content without disclaimers is flagged
- Spam-like content indirectly gets lower scores

Streamlit Application

A simple web interface is built using Streamlit where users can:

- Input URLs
- Analyze content
- View trust scores

This makes the system interactive and user-friendly.

Limitations

- Some websites block scraping
- Metadata is not always available
- Citation count is approximated
- Dynamic content may not load properly

Future Improvements

- Integrate real citation APIs
- Use ML models for trust prediction
- Add summarization using LLMs
- Improve UI with charts and analytics

Conclusion

This project demonstrates how different technologies — web scraping, NLP, and heuristic scoring — can be combined to build a system that evaluates content reliability.

It provides a strong foundation for building more advanced AI systems in the future, such as fake news detection or automated research validation tools.